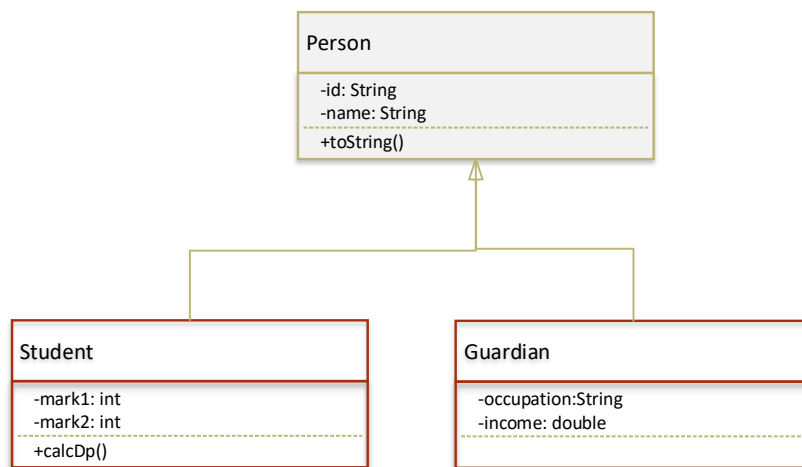

Topic 1- Exercises (Inheritance: Abstract Classes)

We have discussed a special type of Inheritance, in which the super class is an Abstract class. We noted that:

- a. The purpose of an abstract class is inheritance by sub-class
- b. Unlike a concrete class, you cannot create objects (instances) of an abstract class
- c. You can make any class abstract by using the keyword abstract.
- d. However, **as a rule: any class which contains an abstract method** (i.e., method declaration) **MUST** be an abstract class

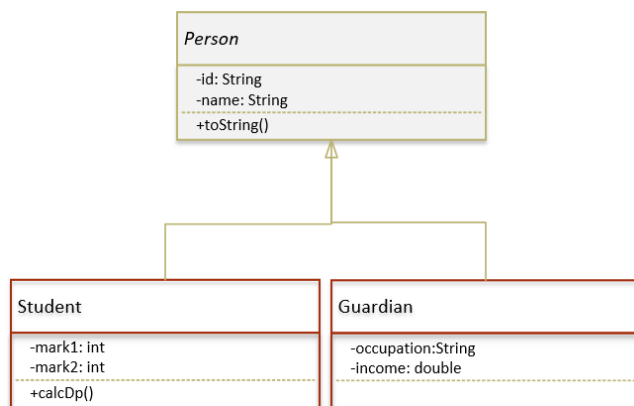
Exercise 1



NOTE: Use ONLY **Person** and **Student** classes in the UML diagram for this exercise

1. Create a package **inherit**.
2. In this package:
 - a. Write the **Person** and **Student** classes (separately). Each class should contain **at least two constructors, get and set, and toString()** methods. **As explained in the lecture, re-use code (constructors, get and set, toString()) in super class to ensure there is no duplication in Student class.**
 - b. Write a **Tester** (driver) which calls parameterized constructors to create and display one instance of **Person** (called **p1**) and **Student** (called **s1**).
 - c. Run the application

Question: The diagram below shows **Person** is an abstract class.



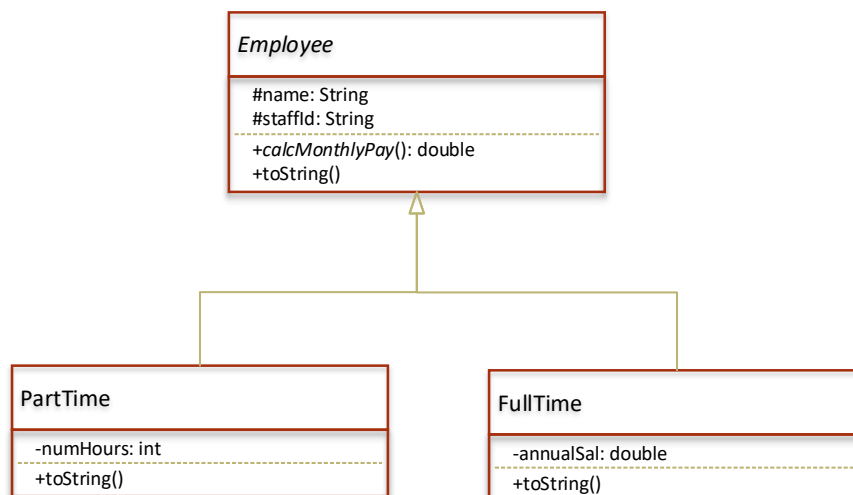
1. Modify the **Person** class to an abstract class. Explain the error in the Tester program.
2. Create a Person instance **p** and assign a Student instance **s**. Why is this correct?

[Exercise 2 \(Answer this question. We'll use the same example in Design patterns](#)

XYL Food is a small business. There are two types of employees: part time and full time. The monthly pay is calculated as follows:

- a. Part time employee: number of hours * rate per hour (assume
- b. Full time employee: annual salary * 1/12

The UML diagram shows the relationship between **Employee**, **PartTime** and **FullTime** classes. The data **name**, **staffId**, **numHours** and **annualSal** represent the name, staff Id, number of hours and annual salary respectively. **calcMonthlyPay()** returns the gross monthly pay as shown in **a** and **b** above.



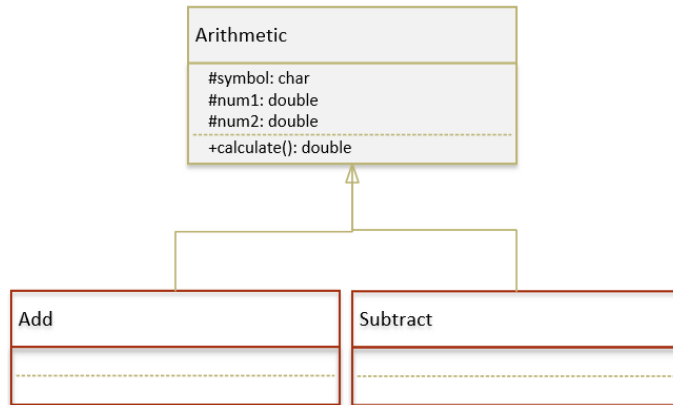
1. Explain why **calcMonthlyPay()** must be declared an abstract method.
2. Write the code for **Employee** class.
3. Write the code **PartTime** class (assume rate per hour is R80.00).

4. Write a Tester to display details of a **PartTime** instance (a typical output is shown below).Id : 20122011 Name: J May

Hours: 20 Rate: R80.0 Monthly pay: R1600.0

5. Create an instance of **Employee**. Explain the error.

Exercise 3 (2 marks MAY be awarded) Answer this question. We'll use the same example for Design patterns



The diagram shows **Add** and **Subtract** inherit the class **Arithmetic**. In **Add**, `calculate()` returns the sum of `num1` and `num2`. In **Subtract**, `calculate()` returns the difference of `num1` and `num2`.

1. Should the method `calculate()` be implemented in **Arithmetic**? Explain your answer?
 2. Based on your answer to (1) above, explain whether or not **Arithmetic** must be declared an abstract class.
 3. Write the method `calculate()` in **Arithmetic**.
 4. Write the method `calculate()` in **Add**
5. Write the code for classes:
1. **Arithmetic**
 2. **Add**
 3. **Tester** which uses the classes **Arithmetic** and **Add** to add two numbers (3 and 10) and display the answer on the screen.